ChatGPT

ECE243 期末项目实施路线图（电子钢琴：PS/2 键盘输入 + 音频合成 + VGA 可视化）

项目目标与范围控制

你们的核心目标可以定义为：在 **DE1-SoC** 上实现一个“可弹奏、低延迟、不掉音、画面反馈清晰”的电子钢琴：用 **PS/2 键盘**（如 `ASDFG...L` 做白键，`WERTY...` 做黑键）触发音符；同时在 **VGA** 上显示一排钢琴键，并在按下/松开时高亮对应琴键。

范围要刻意“收敛”，因为课程项目工作量期望大致相当于约 3.5 个实验规模，并且有明确的里程碑周、演示与最终提交时间点。

建议你们把“像真实乐器一样好”拆成两个可量化的工程指标（这样更容易做取舍）：

第一类是“手感/性能”指标（强烈优先）
按键到出声延迟尽量稳定且足够低；音频连续无明显爆音/断裂；多按键时仍能维持稳定输出。这些是演示时 TA 最容易直观看到的“专业度”。

第二类是“音色逼真度”指标（量力而行）
真实钢琴/吉他音色属于“更难”的目标，但可以用有限成本做到“听起来像个乐器而不是纯蜂鸣”。例如用谐波+包络（ADSR/衰减）做出“钢琴感”的衰减音色，比追求高保真采样要更可控。

核心功能清单与验收标准

为了避免项目变复杂，功能分为“必达（Must-have）”与“加分（Stretch）”，并且每一项都配一个可验收标准。

必达功能（建议作为最终演示的最低配置）
键盘输入：能稳定识别 PS/2 键盘按下与松开事件，而不是只识别字符；并能维护“当前按下的键集合”。PS/2 键盘发送的是扫描码（make/break），break 通常以 `F0` 前缀表示松开，扩展键以 `E0` 前缀出现。 [1]
音频输出：能持续向音频输出 FIFO 写入样本，不出现明显间断。DE1-SoC 课程计算机系统的音频口提供左右声道写 FIFO，并提供可读的 FIFO 空间字段用于“还有多少空间可写”。 [2]
VGA 可视化：VGA 输出支持 640×480，但像素缓冲区分辨率为 320×240（显示时会在 x/y 方向复制放大），适合你们画矩形琴键；字符缓冲区可用于显示文字标签（如音名/模式）。 [3]
联动反馈：按下某键时，VGA 对应琴键在 1 帧内（约 16.7ms）出现高亮；松开则恢复（或进入“释放/衰减”状态）。双缓冲可以在垂直同步时交换，避免撕裂。 [4]

加分功能（做得越多越好，但绝不允许拖垮必达项）
复音（Polyphony）：至少支持 2–4 个同时发声的音符；并且混音不明显失真（可通过缩放/限幅）。
音色切换：用 DE1-SoC 板载开关/按键切换“音色预设”（例如：piano-like / organ-like / guitar-like），以及音量或八度切换。
更“像钢琴”的触感：加入简单包络（快速起音 + 指数衰减）；甚至加入轻微颤音/滤波，让声音更自然。

模块划分与交互方式

为了两人并行、也为了后期集成不崩，建议一开始就把系统拆成 5 个模块，并用“数据结构+事件”连接，而不是互相直接调用硬件。

输入解码模块（PS/2 → KeyEvent）
职责：从 PS/2 端口读扫描码流，运行一个小状态机，把 `make/break/extended` 组合解析成统一事件：`(key_id, pressed/released, timestamp)`。PS/2 口本身有 FIFO，并提供 `RVALID/RAVAIL` 等字段让你判断是否有数据可读、FIFO 里还有多少字节。 [5]
输出：KeyEvent 队列（环形缓冲最稳）。

音符映射模块（KeyEvent → NoteOn/NoteOff）
职责：把 PC 键盘键位映射到音高（Hz 或“相位增量”）。这里要明确你们的“音域范围”：由于系统默认音频采样率是 8k samples/sec，理论奈奎斯特上限是 4kHz；因此更稳妥的策略是把键位映射在中音区/低中音区（比如 C4–C6）而不是最高音区。 [6]
输出：NoteOn/NoteOff 事件 + 当前的 Active Notes 集合。

合成与混音模块（Active Notes → PCM Samples）
职责：每次需要输出一批样本时，根据当前 active voices 生成 PCM。强烈建议用**定点**（fixed-point）与查表（sine wavetable）起步，避免浮点开销和抖动。
输出：写入音频口写 FIFO（Leftdata/Rightdata），并依据 FIFO 空间字段决定一次写多少。 [2]

显示模块（KeyState → VGA Render）
职责：初始化时画静态键盘；运行时只重绘发生变化的键区域（按下高亮/松开恢复），并在需要时用字符缓冲显示当前模式（音色、八度、复音数）。像素缓冲是 320×240、16-bit 565 格式，并可用双缓冲在垂直同步点交换，减少撕裂。 [7]

调度/主循环模块（把实时性“兜住”）
职责：决定谁先谁后。关键原则是：音频是硬实时倾向（不能断），VGA 是软实时（晚一两帧肉眼能忍）。你们可以先用轮询+批量写 FIFO 做到稳定，再视情况上中断：音频端支持在写 FIFO 低于阈值时触发中断（WI/WE 相关位），可以用更少的中断次数保持连续输出。 [8]

主要挑战与对应解决方案

这一部分是你们接下来四周会“真实踩坑”的地方。我按发生概率与破坏性由高到低排序，并给出最省时间的解法。

挑战是 PS/2 并不是 ASCII 键盘输入

很多同学第一次做 PS/2 会卡在：读到的不是字符，而是一串扫描码；同一按键要处理按下（make）和松开（break）；还会遇到 typematic repeat（长按重复 make）。文档明确指出 make/break 的关系：多数 set 2 的 break 是两字节，前缀 `F0` + 该键 make；扩展键还会多 `E0` 前缀。 [9]

最省事的解决方案
把 PS/2 解析写成一个极小状态机（3 个状态就够：Normal / SeenE0 / SeenF0），输出统一事件 KeyEvent。然后“音符映射模块”只认 KeyEvent，不再关心扫描码细节。
另外建议在开发期做一个“调试显示”：在 VGA 字符缓冲区打印最近 4–8 个扫描码十六进制，能极大加速你们定位映射表错误（字符缓冲 80×60，适合显示调试信息）。 [10]

挑战是音频连续性与主循环竞争 CPU

你们的 VGA 画面如果每帧全屏重绘，会非常吃 CPU；而音频如果不能持续向写 FIFO 供样本，就会出现断续、爆音、卡顿。音频口提供写 FIFO，深度可达 128 个 32-bit words，并且有 FIFO 空间寄存器字段告诉你还能写多少。[11]

最省事的解决方案（推荐分阶段）
第一阶段（更快做出来）：轮询 FIFO 空间，采用“批量写入”——每次只要发现 WSLC/WSRC（可写空间）足够，就一次写入一小块样本（比如 32–64 个），从而减少频繁检查。[12]
第二阶段（更稳更像乐器）：启用音频写 FIFO 低水位中断（WE/WI 机制），当写 FIFO 低于阈值触发，你们就在 ISR 里快速填充一批样本。这样音频供给不再被 VGA 绘制“饿死”。[8]

关键注意点
ISR 里不要做复杂绘图；ISR 只做“生成样本+写 FIFO”，其余逻辑（键盘事件处理、UI）放主循环。

挑战是复音混音会溢出与失真

一旦支持多键同时发声，把多个波形直接相加很容易溢出（clipping），听起来会刺耳。你们还会遇到“按得越多越响/越糊”的问题。

实用解决方案
先设定最大复音数（例如 4），并在混音时做简单缩放：`sum / num_voices` 或者总增益固定为 `1/Nmax`（更稳定）。再加一个轻量限幅（clamp）保证不溢出。这样实现成本低、效果立竿见影。
如果想更像钢琴，用“快速 Attack + 指数 Decay”的包络，每个 voice 独立衰减；这比追求真实采样音色更符合 3.5 周的工程现实。

挑战是 VGA 画面撕裂与闪烁

你们如果在显示扫描进行时改像素缓冲，可能出现撕裂。手册说明：像素缓冲控制器支持双缓冲，写 1 到 Buffer 寄存器会在垂直同步时交换前后缓冲（约每 1/60 秒一次），并可轮询状态位判断交换完成。[13]

最省事的解决方案
如果你们的 UI 只是“按下高亮一个键”，其实可以不必每帧重绘：只对发生变化的键区域做局部更新，撕裂概率会很低。
如果你们想把画面做到“像应用一样干净”，再加双缓冲：在 back buffer 画好整帧或局部更新，然后等垂直同步 swap。

挑战是“真实乐器音色”目标容易失控

想做“像真实钢琴/吉他一样”很容易把项目拖爆。尤其是采样回放（sample playback）需要更大的存储、更多的数据搬运，还会引入新的调度难题。

建议的可控路线
把“音色”理解为几个可切换的“合成预设”：
piano-like：基频 + 少量谐波（2–4 个）+ 快速起音 + 指数衰减；
organ-like：谐波更多但衰减更慢；
guitar-like（简化版）：更快的衰减 + 轻微的“拨弦瞬态”（比如起音时加一个短促噪声/高频分量）。
这些都能在“查表 + 定点 + 包络”的框架里完成，不需要引入新环境或新工具。

项目日程与双人并行分工

下面日程以你现在的日期 **2026-03-03（多伦多）** 为起点，并严格对齐课程里程碑：
2026-03-09（周一）获取唯一性审批；2026-03-16（周一）Project Lab #1；2026-03-23（周一）Project Lab #2；2026-03-30（周一）最终演示；最终提交截止是 **2026-04-07（周二）18:00**。

为了效率，我把两位队员分为：
队员 A：音频/合成主责（Audio Lead）
队员 B：键盘/VGA 主责（I/O + UI Lead）
每周至少安排一次“接口冻结点”，强制集成，避免最后一周才拼起来。

预热阶段（2026-03-03 至 2026-03-08）

这一周目标只有一句话：把“必达功能的最小闭环”想清楚并写成接口。

队员 A（音频）
阅读音频口寄存器与 FIFO 机制，确认用“轮询 FIFO 空间 + 批量写入”的最小方案即可启动；明确采样率默认 8k samples/sec 的限制与选定音域。 [14]

队员 B（键盘+VGA）
确定 PS/2 扫描码 set 2 的按下/松开解析策略（`F0` break，`E0` extended）；同时确定 VGA 绘制策略：像素缓冲 320×240（显示 640×480），画矩形键盘即可。 [15]

两人共同
写出“模块接口约定”：KeyEvent、NoteEvent、KeyState、RenderState 的字段；并准备向 TA 解释的 block diagram（你们 Project Lab #1 会被看这个）。

第一阶段（2026-03-09 至 2026-03-15）

本周硬目标：在 2026-03-09（周一）完成“唯一性审批”发帖；并且在 2026-03-16 的 Project Lab #1 前，把系统跑起来一个 MVP。

队员 A（音频）
做“单音”输出：按一个键就出一个稳定音（先用正弦/方波都行），重点是连续、不卡；写 FIFO 时用 Fifospace 判断空间。 [2]

队员 B（键盘+VGA）
把 PS/2 扫描码解析成 KeyEvent，至少支持 `ASDFG...L` 的按下与松开；VGA 画出静态琴键，并实现“收到 KeyEvent 就局部高亮/恢复”。PS/2 口有 FIFO 与可读状态字段，注意及时读走避免堆积。 [16]

集成冻结点（建议 2026-03-13 前）
只要做到“按 A → 出声 + A 对应琴键亮”，就达成 Project Lab #1 的最低展示。带上 block diagram。

第二阶段（2026-03-16 至 2026-03-22）

本周目标：Project Lab #1 当天展示后，把系统变得“可弹奏”，而不是 demo 玩具。

队员 A（音频）
加入包络（最少：attack + decay），让声音更像钢琴；引入 voice 结构，为后续复音做准备；如果轮询方案出现偶发断音，开始评估启用音频中断（写 FIFO 低水位触发）。 [8]

队员 B（键盘+VGA）
完善键盘：处理 typematic（长按重复 make）导致的重复 NoteOn；策略是：如果 key 已处于 pressed，忽略重复 make。PS/2 键盘“长按会重复发送 make，松开才发 break”的行为在资料里有说明。[1]
VGA：加入音名/模式字符显示（字符缓冲 80×60 很适合做状态栏）。[4]

共同集成目标
做到“按住不放持续发声，松开停止/进入衰减”，并且 UI 高亮状态一致。

第三阶段（2026-03-23 至 2026-03-29）

本周目标：为 Project Lab #2 展示“明显进步”，并且把你们想要的“像乐器一样”主要体现在稳定性与复音能力上。

队员 A（音频）
实现 2–4 复音混音 + 缩放/限幅；做至少两种音色预设（piano-like / organ-like），并预留“用板载开关切换”的接口。

队员 B（键盘+VGA）
扩展黑键映射（如 `W E T Y U ...` 对应升号）；UI 上画出黑键并能高亮；如果画面出现撕裂，再引入双缓冲：在 back buffer 画完再垂直同步 swap。[4]

共同集成冻结点（建议 2026-03-26）
功能冻结：从这一天开始，不再新增大功能，只做稳定性与体验优化。这样最后一周不会崩。

第四阶段（2026-03-30 至 2026-04-07）

本阶段目标：2026-03-30（周一）完成最终演示；之后只做“报告与提交物完善”，确保 2026-04-07（周二）18:00 准时提交。

队员 A（音频）
做压力测试：连续弹奏、多人同时按键（快速连按也算），确保无明显断音；微调包络与混音增益，让音量变化自然。

队员 B（键盘+VGA）
做演示脚本：准备几段固定演示（单音阶、和弦、切换音色）；UI 状态栏显示当下音色、八度、复音数，减少 TA 追问时的解释成本。

共同任务
录制视频（课程建议拍视频）；整理提交的代码结构与报告：项目简介、更新后的 block diagram、操作方法、以及工作分工归因表。

测试策略与演示成功要点

为了保证“演示必过”，建议你们采用一个非常工程化的测试清单（每晚 5 分钟跑一遍），重点不是“功能多”，而是“不翻车”。

输入侧测试
快速单击、长按、连按、两键同时按、先按黑键再按白键（以及反过来）都能产生合理事件；并且按下/松开状态不漂移。PS/2 make/break 与 typematic 行为是可预期的，你们的代码只要有明确规则就不会乱。[17]

音频侧测试
持续输出 30–60 秒无断续；复音时无明显爆音；如果启用音频中断，确认中断触发阈值与 FIFO 补给逻辑正确（写 FIFO 低于阈值触发、补满后清除 pending）。[18]

显示侧测试
高亮响应“看起来实时”；若使用双缓冲，确认 swap 在垂直同步完成（文档说明 swap 在一帧末尾进行，并可轮询状态位）。[4]

开发与调试“省环境”原则
如果你们不想在环境上花时间，CPUlator 是一个很好的补充：它在浏览器里模拟系统与 I/O 设备，“无需安装”，适合做早期逻辑调试与回归测试；但最终仍要在真实硬件上验证时序与音频连续性。[19]

---

[1] [9] [15] [17] The PS/2 Keyboard Interface
https://www-ug.eecg.toronto.edu/msl/nios_devices/datasheets/PS2%20Keyboard%20Protocol.htm

[2] [3] [4] [5] [7] [8] [10] [11] [12] [13] [16] [18] Intel® FPGA University Program DE1-SoC Computer Manual
https://ftp.intel.com/Public/Pub/fpgaup/pub/Intel_Material/18.1/Computer_Systems/DE1-SoC/DE1-SoC_Computer_NiosII.pdf

[6] [14] Altera® DE1-SoC Computer with Nios V
https://fpgacademy.org/Downloads/DE1-SoC_Computer_NiosV.pdf

[19] Documentation: CPUlator Computer System Simulator
https://cpulator.01xz.net/doc/